IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Seguimiento de experimentos

Probar muchos modelos **sin perder el hilo**: semillas, registro y test una sola vez

Sprint 15 · S15_01 · 20 min de teoría el 1/12 · cuaderno: 50 min en clase (ejercicios 1 a 3) y material de consulta

LA IDEA · POR QUÉ SE REGISTRA

Al tercer día nadie recuerda qué dio qué

«¿Aquella prueba tan buena la hice yo o Marta? ¿Con qué ajustes? ¿Antes o después de limpiar los datos?»

Un **experimento** es una prueba: un modelo, unos ajustes y una medida. Si no se anota, no se puede repetir ni defender.

1 · Fijar el azar
Semilla y versiones anotadas

→

2 · Anotar cada prueba
Una fila por prueba en un CSV

→

3 · Comparar sin mirar el test
Baseline, validación cruzada y ajustes

→

4 · Test, una sola vez
El examen final del candidato

La rúbrica lo premia: el criterio C (modelado y experimentación) vale 20 de 100 puntos y el nivel «Bueno» pide ajuste de hiperparámetros y registro de experimentos.

LA IDEA · SEMILLAS Y AZAR

Semillas: que el azar se repita

Una **semilla** fija el azar

Partir los datos, mezclarlos o crear un bosque usan azar. Con la misma semilla, el ordenador repite **la misma secuencia**.

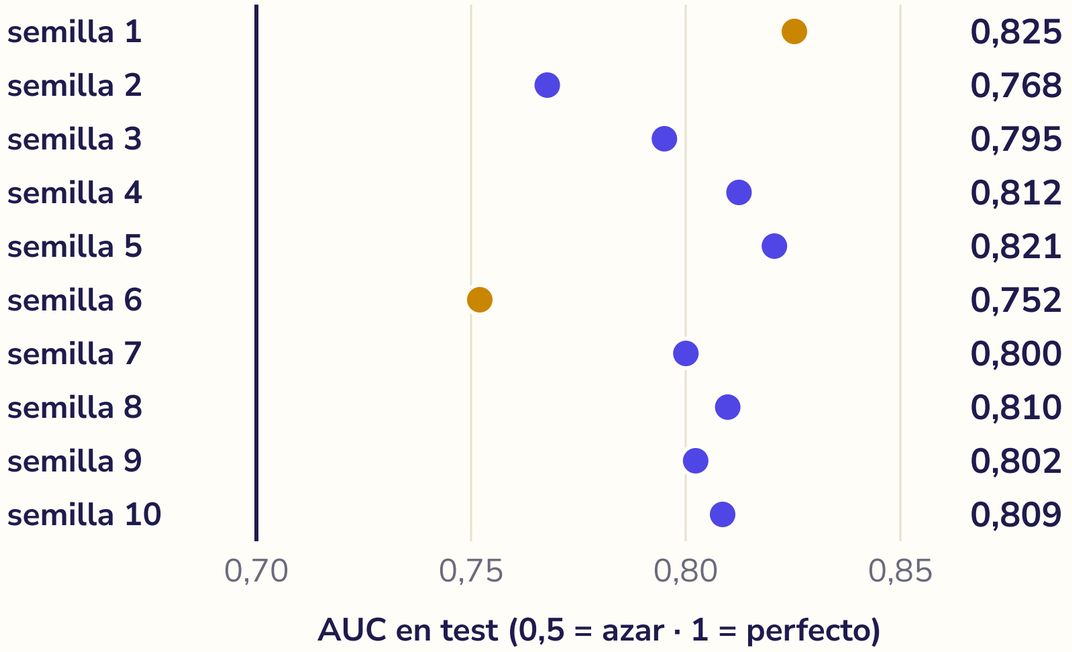
Las **versiones** de las librerías también cambian cifras: se anotan en ENTORNO (python, numpy, pandas y scikit-learn).

Una semilla para todo el equipo

Fijarla (el cuaderno usa SEMILLA = 42) es método.
Cambiarla hasta que salga un número bonito es trampa.

Con la misma semilla y las mismas versiones, otra persona obtiene exactamente tus resultados.

Ejemplo, gimnasio «Marea»: el mismo modelo con 10 repartos



El registro: una fila por prueba

Qué cuenta	Columnas del registro
Cuándo y quién	id, fecha, responsable
Qué se probó	modelo, hiperparametros
Con qué datos	datos_hash (una huella), semilla
Cómo salió	metrica, valor_cv, desv_cv, tiempo_s
Lo que has pensado	notas

Son **12 columnas**, las de la plantilla **registro_experimentos_plantilla.csv**. **valor_cv** y **desv_cv** son la media y la desviación de la validación cruzada (diapositiva 5).

Ejemplo inventado: gimnasio «Marea», se predice la baja de un socio (6 de las 12 columnas).

id	responsable	modelo	valor_cv	desv_cv	notas
1	Ana	Baseline	0,500	0,000	el listón a batir
2	Ana	RegresionLogistica	0,800	0,029	simple y rápida
3	Luis	RandomForest	0,842	0,020	mejor hasta ahora
4	Marta	HistGradientBoosting	0,807	0,027	casi como la logística

Una fila por prueba: lo que no se anota no se puede repetir ni defender.

LA IDEA · COMPARAR SIN MIRAR EL TEST

Validación cruzada: media y desviación

Cinco rondas: cada una mide en el trozo que no ha visto (ejemplo: logística del gimnasio «Marea»)

	trozo 1	trozo 2	trozo 3	trozo 4	trozo 5	
Ronda 1	**mide**	entrena	entrena	entrena	entrena	0,817
Ronda 2	entrena	**mide**	entrena	entrena	entrena	0,759
Ronda 3	entrena	entrena	**mide**	entrena	entrena	0,786
Ronda 4	entrena	entrena	entrena	**mide**	entrena	0,795
Ronda 5	entrena	entrena	entrena	entrena	**mide**	0,845

Media 0,800 · desviación 0,029

En el cuaderno: 5 trozos (**StratifiedKFold**), 4 800 reservas para entrenar y 1 200 de test sin tocar.

Baseline

El modelo más simple, que predice la clase más frecuente. Es el **listón**: su AUC es 0,5.

Test

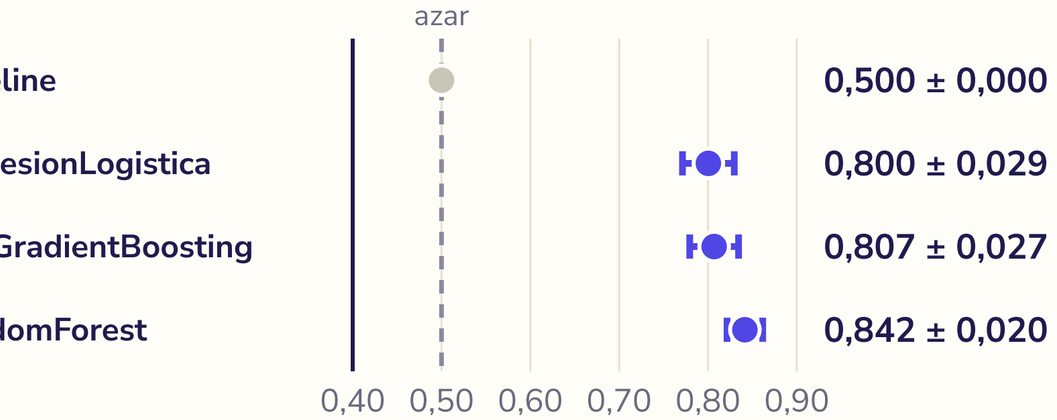
Datos apartados desde el principio. **No se miran** para comparar ni para ajustar: se usan una sola vez, al final.

Regla del ruido

Si dos modelos difieren menos que su desviación, **no hay diferencia real**: elige el más simple, rápido o fácil de explicar y anótalo en **notas**.

Comparar modelos y anotar cada uno

Ejemplo, gimnasio «Marea»: AUC en validación cruzada



Más a la derecha, mejor (barra = ± desviación)

Cómo se lee este ejemplo

Logística y boosting: 0,007 de diferencia, menos que su desviación. Empate: mejor la más simple.

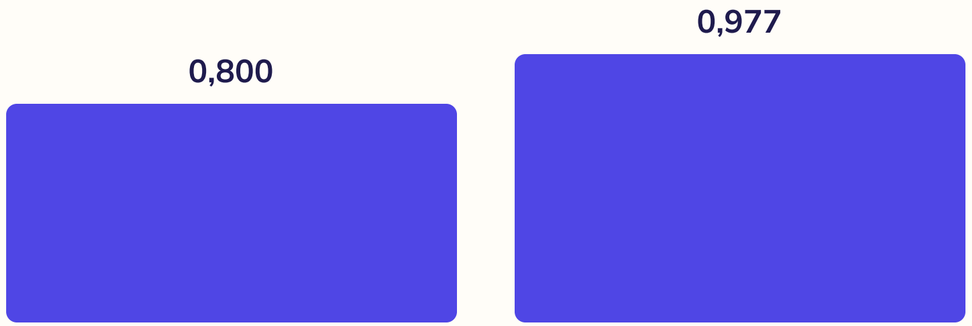
Bosque: 0,035 más que el boosting, más que su desviación; se mira también el tiempo y la explicación.

Se compara con validación cruzada, mirando también el tiempo y la explicación; el test espera al final.

LA IDEA · FUGA DE DATOS

Demasiado bueno para ser verdad

AUC en validación cruzada, mismo modelo · gimnasio «Marea» (ejemplo)



- **¿La conocería el día que decido?** Si no, fuera.
- **Sospecha** de lo demasiado bueno antes de celebrar.

Fuga de datos

El modelo ve una pista que, el día que se use de verdad, **todavía no existiría**.

Aquí **baja_tramitada** solo se rellena **después** de que el socio se dé de baja.

Qué se hace

Se anota con **Fuga** en el nombre y una nota de alarma. Se **descarta**: no puede ser el candidato.

- **Preprocesar dentro de un Pipeline** (cadena de pasos que aprende solo del entrenamiento).
- **En series, partir por fechas**, nunca al azar.

LA IDEA · EVALUACIÓN ÚNICA EN TEST

El test se usa una sola vez

1 · Comparar y ajustar
Con validación cruzada, solo con el entrenamiento

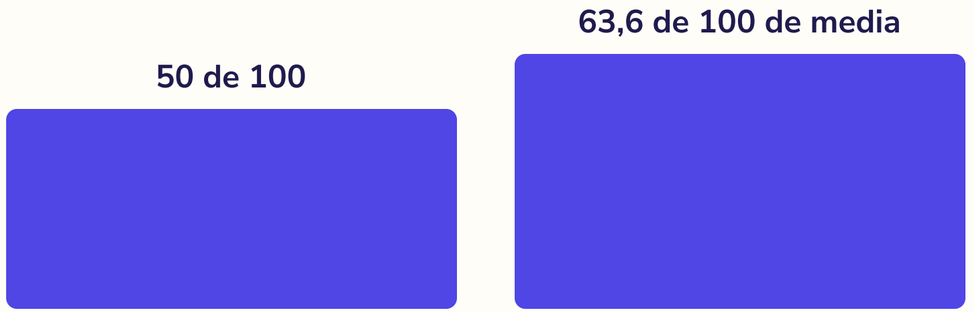
→

2 · Elegir el candidato
Sin mirar el test y sin filas con «Fuga»

→

3 · Test, una vez
El examen final del candidato: se anota y no se vuelve a elegir

Doscientos modelos que **adivinan con una moneda**, medidos en el mismo test de 100 ejemplos:



Si eliges mirando el test

Con tantos intentos, alguno tiene **suerte.** Lo que ves no es un buen modelo: es el que mejor ha adivinado ese test.

Si el resultado decepciona

Se **anota y se explica**. No se vuelve a elegir: un resultado honesto y mediocre vale más que uno bonito con trampa.

LA IDEA · AJUSTAR Y EVALUAR

Ajustar y evaluar el candidato

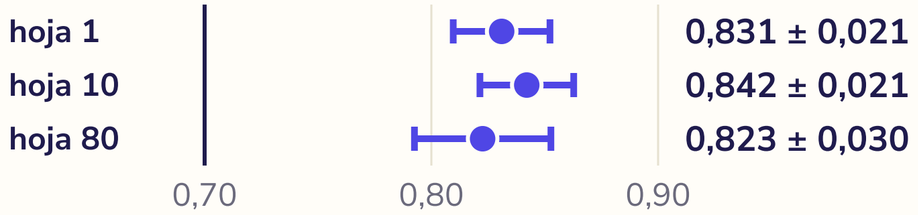
Hiperparámetros

Un **hiperparámetro** es un ajuste que fijas **antes** de entrenar. Cada valor probado es **una fila** del registro.

Validación y test

El candidato se elige con la validación cruzada y **sin** filas con «Fuga»; después se mide **una vez** en el test.

Ejemplo (gimnasio): mínimo de filas por hoja



AUC en validación cruzada (barra = ± desviación)

0,842
Validación cruzada

0,826
Test

Ejemplo del gimnasio: **parecidos, buena señal**. Si el test sale muy por debajo, hay **sobreajuste** (memorizar el entrenamiento) o fuga.

LA IDEA · Y EN VUESTRO PROYECTO

Del cuaderno al proyecto, en tres días

Día	Qué hacéis en el proyecto	Queda anotado
Martes 1/12	Semilla, versiones y registro; baseline y primer modelo mejor (130 min)	Baseline en la primera fila
Miércoles 2/12	Más modelos (clásico, red o LLM/RAG, según el brief) y ajustes; **tutoría técnica de 20 min** por equipo	Una fila de cada persona
Jueves 3/12	Comparativa y elección del candidato (70 min); **test una sola vez** (40 min); entrega en **sprint-15/**	Candidato y resultado en test

Plantilla del registro

registro_experimentos_plantilla.csv trae las **12 columnas** del cuaderno. Se recomiendan **≥ 12 filas** y, al menos, **una de cada persona** (**responsable**).

Hito del sprint

3/12: modelo candidato elegido, sin mirar el test.

Subid de nivel (50 min al día): más experimentos, fugas y registro limpio, comparativa y justificación.

MLflow, mini-reto y preguntas

MLflow (sección 7, opcional)

Herramienta profesional de seguimiento, con interfaz web (**mlflow ui**).

No es obligatoria: el CSV cumple la rúbrica.

Se activa con **USAR_MLFLOW = True** y necesita internet para instalarla.

Mini-reto integrador

Llevad **fijar_semillas**, **registrar** y **experimento** a vuestro proyecto (en el reto, a las primeras celdas de **03_modelos.ipynb**).

Registrad el baseline y al menos 3 modelos con **vuestros datos**.

Cada integrante, **al menos una fila** a su nombre.

Para pensar

1. ¿Qué dos cosas necesita otra persona para reproducir tu mejor resultado?

2. Dos modelos empatan dentro de la desviación: ¿puedes decir que uno es mejor?

3. ¿Por qué no elegir mirando el test?

4. Tu primera prueba da un AUC casi perfecto: ¿qué tres cosas revisas antes de celebrar?

PRÁCTICA · CUADERNO S15_01

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación	Herramientas y ficha del entorno	Clase
Ejercicio 1	Fijar las semillas	Clase
Ejercicio 2	Escribir una fila en el registro	Clase
Ejemplo	Datos, baseline y función **experimento**	Clase
Ejercicio 3	Registrar bosque y boosting y elegir candidato	Clase
Ejemplo	Comparativa de modelos en validación cruzada	Clase
Ejercicio 4	Probar tres profundidades del bosque	Casa
Sección 5	Una variable con fuga de datos	Casa
Ejercicio 5	Evaluar el candidato en el test, una vez	Casa
Cierre	MLflow (opcional), mini-reto y preguntas	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el** ✔. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

Cómo está ordenado

En el **mismo orden que el cuaderno**, cada ejercicio con su código y sus aclaraciones.

Ejercicio	Tema	Diapositivas
1	Fijar las semillas	14 y 15
2	Una fila en el registro	16 y 17
3	Registrar modelos y elegir candidato	18 y 19
4	Tres profundidades del bosque	20 y 21
Sección 5	Una variable con fuga de datos	22
5	Evaluar el candidato en el test	23 y 24

CONSULTA · EJERCICIO 1

Fijar las semillas

```
def fijar_semillas(semilla):
    random.seed(semilla)
    np.random.seed(semilla)
    os.environ["PYTHONHASHSEED"] = str(semilla)
    return np.random.default_rng(semilla)
```

✔ correcto

Salida del cuaderno

un generador

La función devuelve

Una función al inicio del cuaderno basta para que otra persona obtenga exactamente tus resultados.

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer fijar_semillas (ejercicio 1)

random.seed y np.random.seed

Cada biblioteca tiene su **propio generador**: hay que fijarlos **todos**.

str(semilla)

PYTHONHASHSEED es una variable de entorno: solo admite **texto**.

Resultado

No imprime nada; el cuaderno comprueba que dos llamadas dan la **misma secuencia**.

Errores típicos

No guardar el número como texto, olvidar **return** o fijar **solo una** biblioteca.

CONSULTA · EJERCICIO 2

Escribir una fila en el registro

```
def registrar(responsable, modelo, hiperparametros, datos_hash, semilla, metrica, valor_cv, desv_cv, tiempo_s, notas=""):
    existe = os.path.exists(RUTA_REGISTRO)
    n = len(pd.read_csv(RUTA_REGISTRO)) if existe else 0
    fila = pd.DataFrame([{
        "id": n + 1, "fecha": datetime.now().strftime("%Y-%m-%d %H:%M"), "responsable": responsable, "modelo": modelo,
        "hiperparametros": json.dumps(hiperparametros, sort_keys=True), "datos_hash": datos_hash, "semilla": semilla,
        "metrica": metrica, "valor_cv": round(float(valor_cv), 4), "desv_cv": round(float(desv_cv), 4),
        "tiempo_s": round(float(tiempo_s), 2), "notas": notas}], columns=COLUMNAS)
    fila.to_csv(RUTA_REGISTRO, mode="a", header=not existe, index=False)
    return n + 1
```

Salida: ✔ **Ejercicio 2 correcto**. El CSV queda con **12 columnas** y una fila por llamada.

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer registrar (ejercicio 2)

n = len(pd.read_csv(...)) if existe else 0

El **id** es **filas existentes + 1**; sin fichero, empieza en 1.

header=not existe

La cabecera se escribe **solo la primera vez**, cuando el archivo no existía.

Resultado

Cada llamada añade **una fila** al CSV de 12 columnas y devuelve su **id**.

Errores típicos

Repetir la cabecera en cada llamada o abrir el fichero de modo que **borre** las filas.

CONSULTA · EJERCICIO 3

Registrar modelos y elegir candidato

```
experimento("RandomForest", RandomForestClassifier(n_estimators=200, min_samples_leaf=5,
    random_state=SEMILLA, n_jobs=2),
            {"n_estimators": 200, "min_samples_leaf": 5})
experimento("HistGradientBoosting", HistGradientBoostingClassifier(random_state=SEMILLA),
    {"defecto": True})
tabla = pd.read_csv(RUTA_REGISTRO)
candidato = tabla.loc[tabla["valor_cv"].idxmax(), "modelo"]
```

0,755

AUC · regresión logística

0,739 · 0,728

AUC · bosque · boosting

candidato = RegresionLogistica. Desviaciones de **0,011–0,013**; el bosque tarda **3,6 s** frente a **0,14 s**.

CONSULTA · ACLARACIONES · EJERCICIO 3

Leer la comparativa (ejercicio 3)

experimento("RandomForest", ...)

Entrena con validación cruzada, mide el tiempo y **anota la fila** en el registro.

tabla["valor_cv"].idxmax()

Es la posición del **mayor AUC medio**; .loc[..., "modelo"] devuelve su **nombre**.

Resultado

candidato = **RegresionLogistica** (0,755); bosque **0,739** y boosting **0,728**.

Cuidado

Diferencias pequeñas (**0,011–0,013** de desviación): mira también el **tiempo**; y escribe bien los nombres.

CONSULTA · EJERCICIO 4

Probar tres profundidades del bosque

```
resultados_prof = {}
for d in (3, 6, 10):
    media, desv = experimento(f"RandomForest_prof{d}",
        RandomForestClassifier(n_estimators=200, min_samples_leaf=5, max_depth=d, random_state=SEMILLA, n_jobs=2),
        {"n_estimators": 200, "min_samples_leaf": 5, "max_depth": d}, notas="ajuste de profundidad")
    resultados_prof[d] = media
mejor_prof = max(resultados_prof, key=resultados_prof.get)
```

6

Mejor profundidad

0,742 · 0,745 · 0,742

AUC con 3 · 6 · 10

Las tres dan casi lo mismo, **menos que la desviación (≈ 0,013)**: no hay diferencia real; se elige por simplicidad o velocidad.

CONSULTA · ACLARACIONES · EJERCICIO 4

Leer el ajuste (ejercicio 4)

for d in (3, 6, 10)

Cada profundidad es **una prueba** y **una fila** del registro, con nombre distinto: **RandomForest_prof{d}**.

max(resultados_prof, key=resultados_prof.get)

Devuelve la **clave** (la profundidad) con **mayor AUC**.

Resultado

mejor_prof = **6**; **resultados_prof** = {3: 0,742 · 6: 0,745 · 10: 0,742}.

Cuidado

Ejecutar la celda dos veces **duplica filas**: el cuaderno espera **7** en el registro.

CONSULTA · SECCIÓN 5 · UNA VARIABLE CON FUGA

Demasiado bueno para ser verdad

```
X_fuga = X_ent.copy()
X_fuga["reembolso_emitido"] = ((y_ent == 1) & (rng.random(len(y_ent)) < 0.9)).astype(int)
m, d = experimento("LogisticaConFuga", LogisticRegression(max_iter=2000), {"C": 1.0},
                   notas="ALARMA: AUC sospechosamente alto (fuga: reembolso_emitido)", Xd=X_fuga)
mejor_sin_fuga = pd.read_csv(RUTA_REGISTRO).query("modelo != 'LogisticaConFuga'")["valor_cv"].max()
```

0,972
AUC con la fuga

0,755
AUC sin la variable

reembolso_emitido solo existe **después** de cancelar: se anota con «Fuga» y se **descarta**.

CONSULTA · EJERCICIO 5

Evaluar el candidato en el test, una vez

```
tabla = pd.read_csv(RUTA_REGISTRO)
legitimos = tabla[~tabla["modelo"].str.contains("Fuga")]
fila = legitimos.loc[legitimos["valor_cv"].idxmax()]
hp = json.loads(fila["hiperparametros"])
if fila["modelo"] == "RegresionLogistica":
    estimador = LogisticRegression(max_iter=2000)
elif fila["modelo"] == "HistGradientBoosting":
    estimador = HistGradientBoostingClassifier(random_state=SEMILLA)
else:
    estimador = RandomForestClassifier(random_state=SEMILLA, n_jobs=2, **{k: v for k, v in hp.items()})
final = crear_pipeline(estimador, X_ent.columns).fit(X_ent, y_ent)
auc_test = roc_auc_score(y_test, final.predict_proba(X_test)[:, 1])
auc_cv_candidato = fila["valor_cv"]
```

AUC en validación **0,755** y en test **0,745**: **candidato** = RegresionLogistica, resultado realista.

CONSULTA · ACLARACIONES · EJERCICIO 5

Leer la evaluación en el test (ejercicio 5)

~tabla["modelo"].str.contains("Fuga")

Deja fuera las filas con **«Fuga»** en el nombre: no pueden ser el candidato.

crear_pipeline(...).fit(X_ent, y_ent)

Reentrena el candidato con **todo el entrenamiento**; el test se usa **una vez**, después.

Resultado

auc_cv_candidato = **0,755** y auc_test = **0,745**: parecidos.

Cuidado

Si el test sale muy por debajo: **sobreajuste** o fuga; y no se vuelve a elegir mirando el test.